ClipsAI Pipeline

> Pipeline automático para transformar vídeos longos do YouTube em Shorts virais e publicar no YouTube, TikTok e Instagram Reels.

Como funciona

```
URL do YouTube
     │
     ▼
  Download ──────────────────── pytubefix + yt-dlp (OAuth2)
     │
     ▼
 Transcrição ─────────────────── WhisperX (GPU/CPU)
     │
     ▼
Detecção de clips ─────────────── ClipsAI (embeddings semânticos)
     │
     ▼
  Exportação vertical 1080×1920
     ├── Legendas karaoke word-by-word
     ├── Face tracking / crop dinâmico
     ├── Title card animado (fade in/out)
     └── Música de fundo automática (Jamendo CC)
     │
     ▼
Análise LLM ────────────────── Groq (título · legenda · hashtags · score
viral)
     │
     ▼
  Publicação ──────────────────── Yo
```

Requisitos do sistema

Ferramenta	Versão	Observação
Python	3.13	Versão testada e recomendada
Node.js	20+ (LTS)	Necessário para yt-dlp resolver challenges do YouTube
ffmpeg	recente	Precisa estar no PATH do Windows
GPU NVIDIA	CUDA 12.x	Recomendado — CPU funciona mas é muito lento
CUDA Toolkit	12.6	Compatível com os wheels PyTorch usados

1. Variáveis de ambiente do Windows (PATH)

Antes de qualquer coisa, garanta que estas ferramentas estão no PATH do Windows.

Verificar o que já está instalado

Abra o PowerShell e rode:

```
python --version     # deve retornar Python 3.13.x
node --version       # deve retornar v20.x.x ou superior
ffmpeg -version      # deve mostrar versão do ffmpeg
nvidia-smi           # mostra a GPU e versão do CUDA
```

Instalar Python 3.13

Baixe em python.org/downloads → versão **3.13.x**

Durante a instalação:

- ☑ Marque **"Add Python to PATH"**
- ☑ Marque **"Install for all users"** (recomendado)

Instalar Node.js

Baixe em nodejs.org → **LTS (v20+)**

O instalador adiciona automaticamente ao PATH.

Instalar ffmpeg

1. Baixe em gyan.dev/ffmpeg/builds → `ffmpeg-release-essentials.zip`
2. Extraia para `C:\ffmpeg\`
3. Adicione `C:\ffmpeg\bin` ao PATH:

```
# Rodar como Administrador no PowerShell:
[System.Environment]::SetEnvironmentVariable(
    "Path",
    [System.Environment]::GetEnvironmentVariable("Path","Machine") +
";C:\ffmpeg\bin",
    "Machine"
)
```

Ou: `Win + R` → `sysdm.cpl` → **Avançado** → **Variáveis de Ambiente** → `Path` → **Editar** → **Novo** → `C:\ffmpeg\bin`

Feche e reabra o PowerShell após adicionar ao PATH.

---

2. Clonar o projeto

```
git clone https://github.com/SEU_USUARIO/clipsai-pipeline.git
cd clipsai-pipeline
```

---

3. Criar o ambiente virtual

```
# Cria o venv com Python 3.13
python -m venv .venv

# Ativa o venv (necessário em cada sessão nova)
.venv\Scripts\activate
```

> Após ativar, o terminal mostra `(.venv)` no início da linha.

---

4. Instalar PyTorch com CUDA

> ⚠ **Instale o PyTorch ANTES do `requirements.txt`** — a versão depende da sua GPU.

Verifique sua versão CUDA com `nvidia-smi` (coluna "CUDA Version").

CUDA 12.6 (recomendado — testado):

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu126
```

CUDA 12.1:

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121
```

Sem GPU (CPU only):

```
pip install torch torchvision torchaudio
```

Verifique se instalou corretamente:

```
```

```
python -c "import torch; print(torch.__version__); print('CUDA:',
torch.cuda.is_available())"
```

---

5. Instalar dependências do projeto

```
pip install -r requirements.txt
```

---

6. Configurar credenciais

YouTube — download de vídeos (cookies)

O YouTube exige autenticação para evitar bloqueio por bot detection.

1. Instale a extensão Chrome **"Get cookies.txt LOCALLY"**
2. Abra youtube.com **logado na sua conta**
3. Clique na extensão → **Export** → salve como `cookies.txt` na raiz do projeto

> ⚠ Os cookies expiram com o tempo. Re-exporte quando aparecer erro de autenticação.

Alternativa com OAuth2 (mais estável):

```
.venv\Scripts\yt-dlp --username oauth2 --password ""
"https://www.youtube.com/watch?v=dQw4w9WgXcQ"
# Abre uma URL para autorizar no browser — token salvo localmente
```

YouTube — upload de Shorts (API)

1. Acesse console.cloud.google.com
2. Crie um projeto → ative a **YouTube Data API v3**
3. Crie credenciais: **OAuth 2.0** → **Aplicativo para computador**
4. Baixe e salve como `client_secrets.json` na raiz do projeto
5. Na primeira execução do `post_clips.py`, um browser abre para autorizar → token salvo em `youtube_token.json`

Groq — análise LLM (títulos, hashtags, score viral)

Crie uma conta gratuita em groq.com e gere uma API key.
Passe via `--groq-key "gsk_..."` ou configure como variável de ambiente:

```
# Adicionar permanentemente às variáveis de usuário (PowerShell como admin):
[System.Environment]::SetEnvironmentVariable("GROQ_API_KEY", "gsk_...", "User")
```

7. Uso

Processar um único vídeo

```
.venv\Scripts\python run_clipsai.py `
  --url "https://www.youtube.com/watch?v=VIDEO_ID" `
  --out_dir output\VIDEO_ID `
  --device cuda `
  --model small `
  --vertical 1080x1920 `
  --karaoke --karaoke-style red --karaoke-position center `
  --title-card `
  --dynamic-crop `
  --llm-enhance --groq-key "gsk_..." `
  --cookies cookies.txt
```

Postar no YouTube

```
.venv\Scripts\python post_clips.py "output\VIDEO_ID\*.clips.json" `
  --youtube `
  --min-score 7
```

Batch automático — múltiplos vídeos

Crie `urls.txt` com uma URL por linha:

```
https://www.youtube.com/watch?v=ID1
https://www.youtube.com/watch?v=ID2
# linhas com # são ignoradas
```

Execute:

```
.venv\Scripts\python batch_pipeline.py urls.txt `
  --youtube `
  --min-score 7 `
  --karaoke --karaoke-style red --karaoke-position center `
  --title-card `
  --dynamic-crop `
  --llm-enhance --groq-key "gsk_..." `
  --auto-bgm `
  --cookies cookies.txt
```

O progresso é salvo em `batch_state.json` — **interrompa e retome sem reprocessar**.

Flags de referência

`run_clipsai.py`

Flag	Padrão	Descrição
`--url`	—	URL do YouTube a processar
`--input`	—	Arquivo de vídeo local (pula o download)
`--out_dir`	`output`	Pasta de saída
`--device`	`cpu`	`cuda` para GPU, `cpu` para CPU
`--model`	`small`	Modelo Whisper: `tiny` · `small` · `medium` · `large`
`--vertical`	—	`1080x1920` para formato vertical Shorts
`--karaoke`	off	Legendas word-by-word estilo TikTok
`--karaoke-style`	`yellow`	`yellow` · `white` · `red` · `green`
`--karaoke-position`	`bottom`	`bottom` · `center` · `top`
`--title-card`	off	Título animado nos primeiros 2s
`--dynamic-crop`	off	Face tracking automático no crop
`--llm-enhance`	off	Gera título/legenda/hashtags com Groq
`--groq-key`	—	Chave da API Groq
`--bgm-dir`	—	Pasta com músicas de fundo (.mp3/.wav)
`--cookies`	—	Arquivo `cookies.txt` do YouTube
`--cache-dir`	—	Cache de vídeos baixados (evita re-download)

`batch_pipeline.py` (flags adicionais)

Flag	Descrição
`--auto-bgm`	Baixa música adequada ao tema automaticamente via Jamendo (CC)
`--min-score N`	Só posta clips com score de viralidade ≥ N (0–10)
`--youtube`	Posta no YouTube Shorts após clipar
`--instagram`	Posta no Instagram Reels após clipar
`--dry-run`	Mostra os comandos sem executar nada
`--state-file`	Caminho customizado para o arquivo de estado

Estrutura do projeto

```
clipsai-pipeline/
│
├── run_clipsai.py        # Pipeline principal
├── post_clips.py         # Upload YouTube / Instagram
├── batch_pipeline.py     # Orquestrador batch
├── enhance_clips.py      # Roda só o LLM em clips já processados
├── auto_bgm.py           # Download automático de música (Jamendo CC)
│
├── requirements.txt      # Dependências pip (sem PyTorch — instalar separado)
├── urls.txt              # Lista de URLs para o batch
│
├── output/               # (gerado — não versionado)
│   └── VIDEO_ID/
│       ├── clip000.mp4
│       ├── clip000.jpg   ← thumbnail
│       ├── clip000.ass   ← legendas karaoke
│       └── *.clips.json  ← metadados + análise LLM
│
├── bgm/                  # (gerado — não versionado)
│
# Credenciais — NÃO versionar (estão no .gitignore):
├── client_secrets.json
├── youtube_token.json
├── cookies.txt
└── .env
```

Solução de problemas

`n challenge solving failed` (yt-dlp)

Node.js não está instalado ou não está no PATH.

```
node --version   # deve retornar v20+
pip install yt-dlp-ejs
```

`Sign in to confirm you're not a bot`

Cookies expirados. Re-exporte o `cookies.txt` do Chrome (veja seção 6).

`player_ias.vflset` (pytubefix)

```
pip install -U pytubefix
```

`mediapipe has no attribute 'solutions'`

mediapipe 0.10+ removeu o módulo `solutions`. O pipeline usa OpenCV como fallback — **nenhuma ação necessária**.

`uploadLimitExceeded` (YouTube)

Limite diário de uploads atingido. Acesse youtube.com/verify.
O limite reseta à meia-noite PT (≈ 4h BRT).

`paging file too small` (Windows)

Aumente a memória virtual:
`Win+R` → `sysdm.cpl` → **Avançado** → **Desempenho** → **Configurações** → **Avançado** → **Memória Virtual**
Configure: inicial **8192 MB** · máximo **16384 MB** → reinicie o PC.

Música automática

Com `--auto-bgm`, as faixas são baixadas do Jamendo sob licença **Creative Commons**.
Consulte a licença específica de cada faixa em jamendo.com antes de monetizar.